Ticket To Ride Design Document

Authors:
Ng Zi Wei (33590877)
Hew Jia Jing (33589739)
Yu Jia Ming (33592586)

Team: Solution Seekers

Document Revision History

Date	Version	Description	Author
9/3/2026	0.0	Add initial introduction, team membership, team schedule, technology justification	Hew Jia Jing
12/3/2026	0.1	Add initial concept description and scenario description	Hew Jia Jing, Yu Jia Ming, Ng Zi Wei
15/3/2026	0.2	Improve on domain model concept description, and add assumption	Hew Jia Jing
16/3/2026	0.3	Improve on the structure of technology stack and justification	Ng Zi Wei, Yu Jia Ming
18/3/2026	0.4	Add justification to concept description	Ng Zi Wei
23/3/2026	0.5	Improve introduction and team introduction section to include problem statement and tech tools used	Hew Jia Jing, Ng Zi Wei
24/3/2026	0.6	Add team photo and complete on the team introduction section	Ng Zi Wei, Hew Jia Jing, Yu Jia Ming
25/3/2026	0.7	Improve the sequence diagram scenario description	Yu Jia Ming
26/3/2026	0.8	Improved on sequence diagram scenario description, domain model rationale and concept description	Ng Zi Wei, Hew Jia Jing, Yu Jia Ming
27/3/2026	0.9	Improved on sequence diagram scenario description, discarded and final domain model rationale and concept description	Ng Zi Wei, Hew Jia Jing, Yu Jia Ming

Table Contents

1 Introduction 3
2 Team Introduction 5
2.1 Team Name and Team Photo 5
2.2 Team Membership 5
2.3 Team Schedule 6
2.4 Technology Stack and Justification 7
Programming Language: Java 7
GUI Framework: Java Swing 8
Alternative Technology Considered for Programming Language & GUI Framework 8
Team Expertise Mapping 9
Support required 10
Final Decision 10
Technology Prototyping 10
2.5 Link to the Gitlab repository 11
3 Domain Modelling 11
3.1 Domain Model 11
3.1.1 Discarded Domain Models 11
3.1.2 Final Domain Model Chosen 18
3.2 Concept Descriptions 19
Game Core Concepts 19
Board Structure 22
Player Components 25
Cards and Decks 26
Special Components 32
3.3 Assumptions and Domain Constraints 33
4 Dynamic Model 35
4.1 Scenario: Claim a Route (Standard Turn) 35
4.2 Scenario: End-game Final Scoring and Winner Selection 37
5 Supplementary Documentation 39
5.1 Additional Documentation for Scenario 4.2 39
5.2 Contributor Analysis 40
6 AI Declaration 40

1 Introduction

This document aims to provide a comprehensive overview of the technical design and architecture, allowing the development team to develop the system based on it. The project focuses on implementing the Ticket To Ride: North America Edition board game as a 2D standalone application.

The main problem addressed in this project is to translate a complex physical board game into a digital format, while maintaining its official rules. The system must handle core gameplay elements including turn management, train and destination card distribution, route-claiming validation and end-game scoring, delivered through an interactive graphical interface.

This document is structured into the following sections:

- **Section 2: Team Introduction:** Overview of the development team, roles and responsibilities, team schedule, selected technology stack, and justification for each.

- **Section 3: Domain Modelling:** A structural representation of the problem space, including key entities, relationships and design assumptions made.

- **Section 4: Dynamic Model:** Interaction diagrams illustrating key game logic.

- **Section 5: Supplementary Documentation:** Additional resources and references.

This document should be referenced together with the Ticket To Ride Rule book, as referenced in Section 5.

2 Team Introduction

2.1 Team Name and Team Photo

Personal Team Name: Solution Seekers

Moodle Team Name Mapping: This team corresponds to MA_Friday02PM_08 in the Moodle page.

Team Photo:

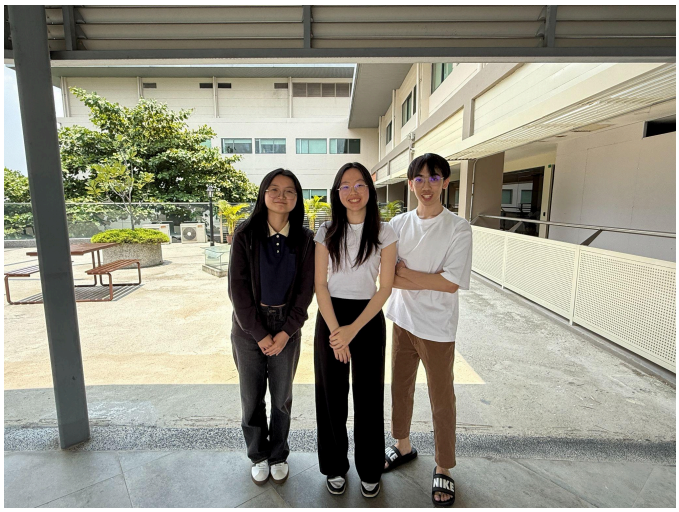


Figure 1: Team Photo
From left to right: Ng Zi Wei, Hew Jia Jing, Yu Jia Ming

2.2 Team Membership

Name	Email	Role	Strengths	Fun Fact
Ng Zi Wei	zngg0041@student.monash.edu	Team Lead, Developer	Object-oriented design, debugging, code quality awareness, attention to detail, collaborative teamwork	Enjoys late-night coding sessions while listening to music.
Hew Jia Jing	jhew0010@student.monash.edu	Developer	Problem-solving,bug tracking, collaborative coding, UI design	I stay awake with tea, not coffee.
Yu Jia Ming	jyuu0107@stu	Developer	Object-oriented	I stay awake with

	dent.monash.edu		design, problem solving, debugging	coding, not coffee nor tea

Roles and Responsibilities

Team Lead
The team lead is responsible to coordinate the overall project progress, manage workload distribution and facilitate communication among team members. The team lead also needs to monitor deadlines and ensure all deliverables meet the requirements quality standard.

Developer
All team members are responsible for the implementation of the system, including coding, system designing, UI designing, debugging, executing unit and integration tests, maintaining consistent code style, and actively contributing in discussions to ensure consistency and quality across the project.

2.3 Team Schedule

Meeting Schedule:

Our team decided to hold meetings according to the scheduled below:

Meeting Type	Day & Time	Host	Purpose
Sprint Workshop	At the start and end of sprint	In-person	Deliverable review and sprint planning
Weekly Stand up	Three times per week (Mon/Wed/Fri)	Zoom/WhatsApp	Progress updates and blockers
Ad hoc session	If required	Zoom/WhatsApp	Bug fixes and urgent discussions

All members are expected to attend the scheduled sessions. If team members face urgent issues that need to be excused, team members are required to inform the team in advance to hold accountability. Communication between meetings is primarily conducted through WhatsApp to ensure timely updates.

Weekly stand-up meetings are typically held on Monday (12:00 pm), Wednesday (12:00 pm), and Friday (2:00 pm), with each session lasting approximately 15-20 minutes.

Regular Work Schedule:

Each team member is expected to dedicate approximately 7 hours per week (averaging 1 hour per day) on individual work. This flexible schedule is designed to accommodate individual timetables while maintaining consistent project progress.

During these individual sessions, team members are expected to focus on their assigned technical tasks, including writing and refining code, feature testing, debugging, and updating documentation. This time is also used to prepare feedback for upcoming group meetings. This approach ensures that individual tasks are properly progressed, allowing all members to make meaningful contributions to the project.

Workload distribution:

The team manages workload using Trello, where tasks are created as User Stories representing specific tasks. During Sprint Planning, the tasks are reviewed, refined, and assigned story points based on their estimated complexity. This process ensures the work is divided fairly so that no one is overwhelmed. Task dependencies are also discussed during the sprint planning session to ensure all team members have a clear understanding of the requirements before feature implementation begins.

Tasks are assigned to individual members one at a time, based on capabilities and current workload. Members may take additional tasks once prior tasks are complete. The team ensures that total story points are fairly distributed among members, and workload redistribution only occurs if a member reports difficulty or if a task is too large to complete individually.

Task Tracking Tool: Trello
Task progress is tracked on Trello using statuses including “To Do”, “In Progress” and “Completed”. Task progress is also discussed during regular stand-up meetings, ensures clear communication, prevents misunderstandings about task requirements, and allows the team to address blockers promptly.
Link to Trello board:
https://trello.com/invite/b/69c670b20c8a262cd618e698/ATTI7e50dc57108bbeceaab9855dabc728f2A07A1AE8/fit3077

2.4 Technology Stack and Justification

Programming Language: Java

Our team has selected Java as our primary programming language for this project, as all three members of the team are familiar with it from previous coursework (FIT2099, FIT1051) and internship experience where object-oriented programming concepts including encapsulation, polymorphism and inheritance were extensively applied, hence

no member will need a significant learning curve. Moreover, Java has a strong object-oriented nature which fits the game development of the Ticket To Ride game, where game entities, such as players, routes, train cards and destination cards can be cleanly modelled as classes and objects. Besides, the static typing and compile time error checking of Java reduces the likelihood of runtime errors and improves debugging efficiency when developing a complex system collaboratively.

GUI Framework: Java Swing

We selected Java Swing as the project's graphical user interface framework, as it is a standard Java UI toolkit included in Java's SDK. Hence, interactive UI controls and rendering can be done without additional libraries required. This allows the team to focus on development without the overhead of configuring new tools or environments. However, all three team members have no prior experience with it, but the team decided that a simple tutorial will be taken prior implementation, hence a learning curve will still be expected. Also, since all three team members already have strong familiarity with Java, learning Java Swing involves extending existing Java knowledge instead of adopting a completely new programming ecosystem.

Alternative Technology Considered for Programming Language & GUI Framework

Python with TKinter was considered as an alternative technology stack for this project. Python offers simple syntax and faster prototyping, which can be beneficial for rapid development. Nevertheless, the team's previous experience with Python has primarily focused on algorithm and data structure implementations instead of building object-oriented software systems. Thus, using Python in implementing the system would introduce a steeper learning curve in areas such as object-oriented architecture, project structuring and GUI integration. Moreover, the team has no experience using TKinter to develop graphical applications, so adopting both a new programming environment and a new GUI framework could introduce higher development risks within the project timeline.

Development and Documentation Tools

Diagram drawing tools choices: Lucid Chart and draw.io

The team decided to use *Lucid Chart* to create any diagrams required, including domain models and UML interaction diagrams, as it is accessible, provides a clear interface, and supports collaborative editing. More importantly, all of the team members had more experience with Lucid Chart compared to draw.io as the team had been using it consistently across the past academic studies.

UI and visualization planning: Figma

The team decided to use Figma for UI planning and visual layout explorations. Figma is well suited for simple mock-ups before implementation, its ease of use and collaborative features make it suitable for early-stage design. This allows the team to communicate

design ideas clearly. Additionally, each of the team members has experience in using Figma across the past projects.

Build, Run and Development Approach

The application will be developed as a standalone Java application using the Intellij IDEA as the primary development environment, as the team is familiar with it and it provides integrated support for building, running, and packaging the project.

During development, the application will be compiled and executed using standard Java tools to ensure consistent functionality. The project will be packaged into an executable JAR file, allowing the application to run independently outside the development environment.

Early testing of the build and execution process will be conducted to ensure the JAR file can be successfully executed without additional setup. Clear introduction will also be provided to guide users on how to run the application locally.

Version Control and Collaboration

The team will use the official Monash Github repository for version control and collaboration. All members are required to commit their work regularly to maintain a consistent contribution history and tracking purposes.

Team members are expected to follow the following Git policies:

- Each member needs to create and work on a personal branch for their assigned tasks, instead of working directly on the main branch, to keep the main branch stable at all times.
- Merge requests must be created to merge the member personal branches into the main branch, and at least one team member other than the branch owner should review the changes before approval to avoid potential issues.
- Team members should pull the latest updates frequently and always pull before pushing their own changes to minimize the merging conflict and ensure they are working with the latest codebase.
- All commits must include a meaningful commit message that describes the changes made.
- Each commit should be kept as small and focused as possible but not include different types of task or unrelated changes in a single commit.

Team Expertise Mapping

The following table summaries the current technical expertise relevant to the selected tech stack of our team.

Skill Area	Team Experience
Java Programming	Having intermediate experience from previous coursework

	and internship
Object-oriented Design	Familiar with core OOP principles including SOLID concepts, polymorphism, interface, inheritance and class design.
Version Control (Git)	Experienced in collaborative development using Git for version control and code integration.
GUI Development	No prior experience with Java Swing. Previous UI exposure is limited to HTML based interfaces and Android XML layouts.
Project Management (Trello)	Familiar with Agile and Kanban practices using Trello for sprint planning and task tracking from previous coursework.
Documentation Tool (LucidChart)	Experienced in drawing uml class diagrams and sequence diagrams from previous coursework (FIT2099).
UI Planning (Figma)	Familiar with developing high-fidelity prototypes using Figma.
IDE (IntelliJ IDEA)	Experienced with local Java development in IntelliJ IDEA, including plugins, debugging tools and IDE settings.

Support required

The team potentially needs support from the teaching team in Java Swing advanced layout management and development, as none of the team members have previously developed a full graphical application using Swing, support in best practices and debugging UI-related issues will help reduce development risks.

Final Decision

In conclusion, our team has selected Java with Java Swing as the final technology stack for the implementation of the Ticket to Ride game system based on the existing expertise of our team in Java as well as the alignment with project requirements. This choice minimizes learning overhead, reduces development risk, and supports a more maintainable implementation of the system.

Supporting tools including IntelliJ IDEA, LucidChart, Figma, and Trello were selected based on existing team familiarity, requiring no additional justifications.

Technology Prototyping

Each team member has created a basic Java technology prototype to verify that we can write and compile Java code locally, and will not need to rely heavily on each other on finishing technology tasks in the future sprints. Prototypes are kept in the /src/prototype directory of the team's GitLab repository, in individual sub-folders named by team members. Each sub-folder contains the source code written by the member with a

README file (this README contains the instructions of how the JAR file is created and how to run it), and the compiled JAR files are stored under /dist directory.

Source code directory: /src/prototype

JAR file directory: /dist

2.5 Link to the Gitlab repository

https://git.infotech.monash.edu/FIT3077/fit3077-s1-2026/assignment-groups/MA_Friday02PM_08

3 Domain Modelling

3.1 Domain Model

3.1.1 Discarded Domain Models

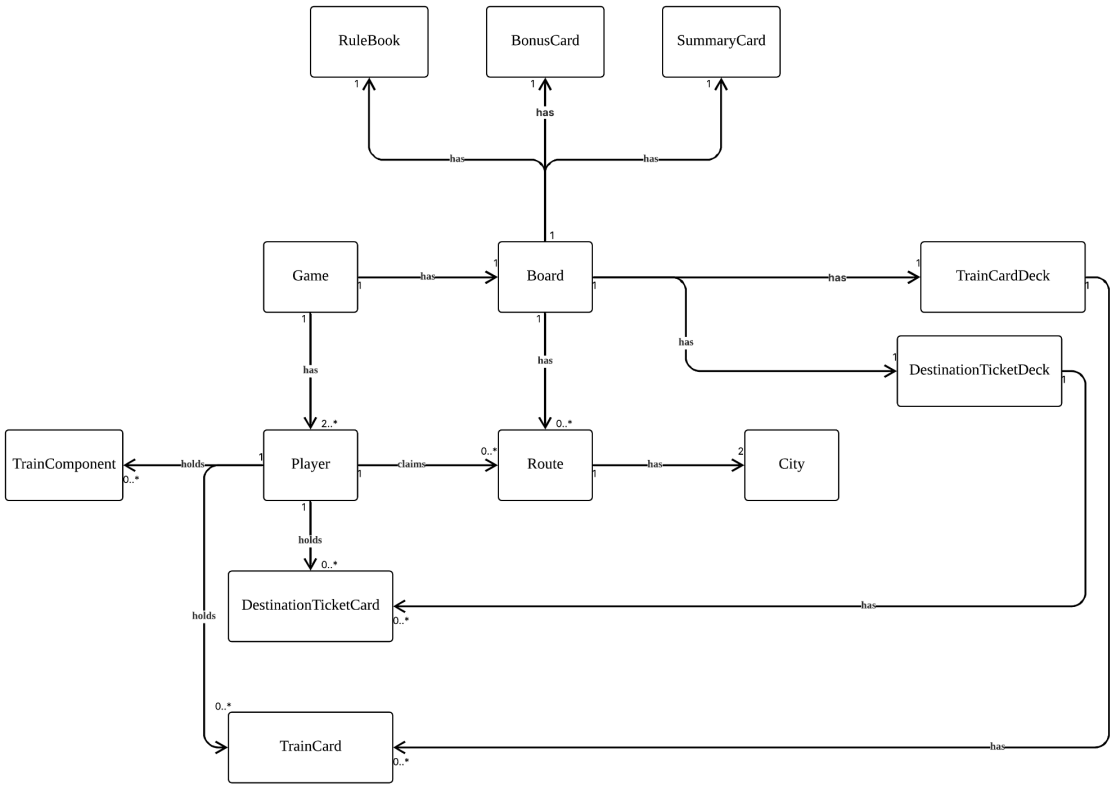


Figure 2: Discarded Domain Model Iteration 1

Alternative Logic:

In this version, Route were connected directly to Board, and City were only associated with Route. All relationships were modelled using simple associations. This version simplifies the model by reducing the number of entities and relationships, making the model easier to construct and read.

Rationale for Discarding:

1. **Inability to express ownership**: Using only simple associations for all relationships fails to show stronger structural relationships, such as ownership. For example, the relationship between Game and Player can be shown as a stronger structural relationship as Player only exists if there is a Game to play.

2. **Ignoring domain constraints in cardinalities**: This model uses generic cardinalities like 2..* (Game to Player), 0..* (Player to TrainComponents) which do not correctly reflect the game domain constraints. For instance, the number of Players is bounded, and TrainComponents are limited by physical game pieces.

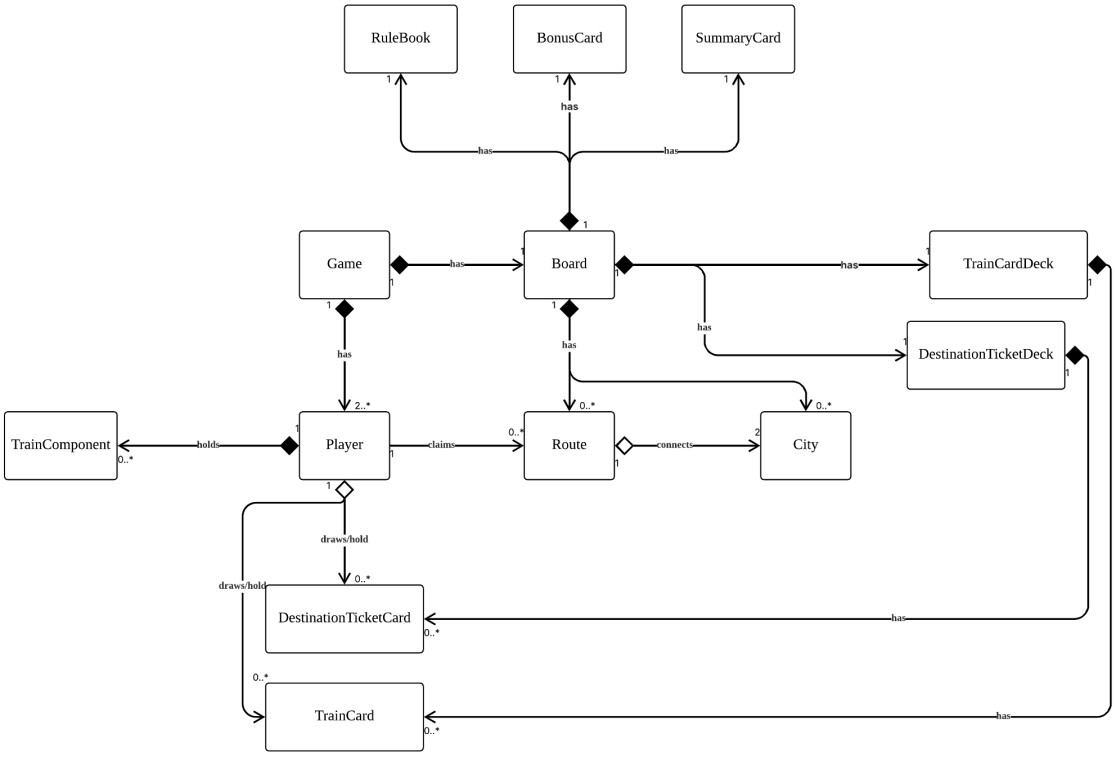


Figure 3: Discarded Domain Model Iteration 2

Alternative Logic:

This version extends Alternative 1 by introducing aggregation and composition relationships, showing stronger structural relationships between concepts. Game is composed of Players and a Board. Board composes core components such as TrainCardDeck, DestinationTicketDeck, Route and City. Player has aggregation with both DestinationTicketCard and TrainCard, and composition with TrainComponents. This design improves clarity by introducing stronger relationship types, making ownership and structure between entities more explicit. It also improves accuracy by linking Board to City, showing correct board configuration.

Rationale for Discarding:

1. **Overuse/misuse of aggregation/composition**: Some relationships (e.g. between Route and City) are wrongly modelled using aggregation, which may incorrectly imply ownership, as Cities do not belong to Routes, they are only connected through them. This misuse of relationship will reflect in inconsistent game configuration within the game session.

2. **Unclear ownership boundaries**: While composition is introduced, some relationships may become overly restrictive or too tightly coupled, reducing flexibility in the model.

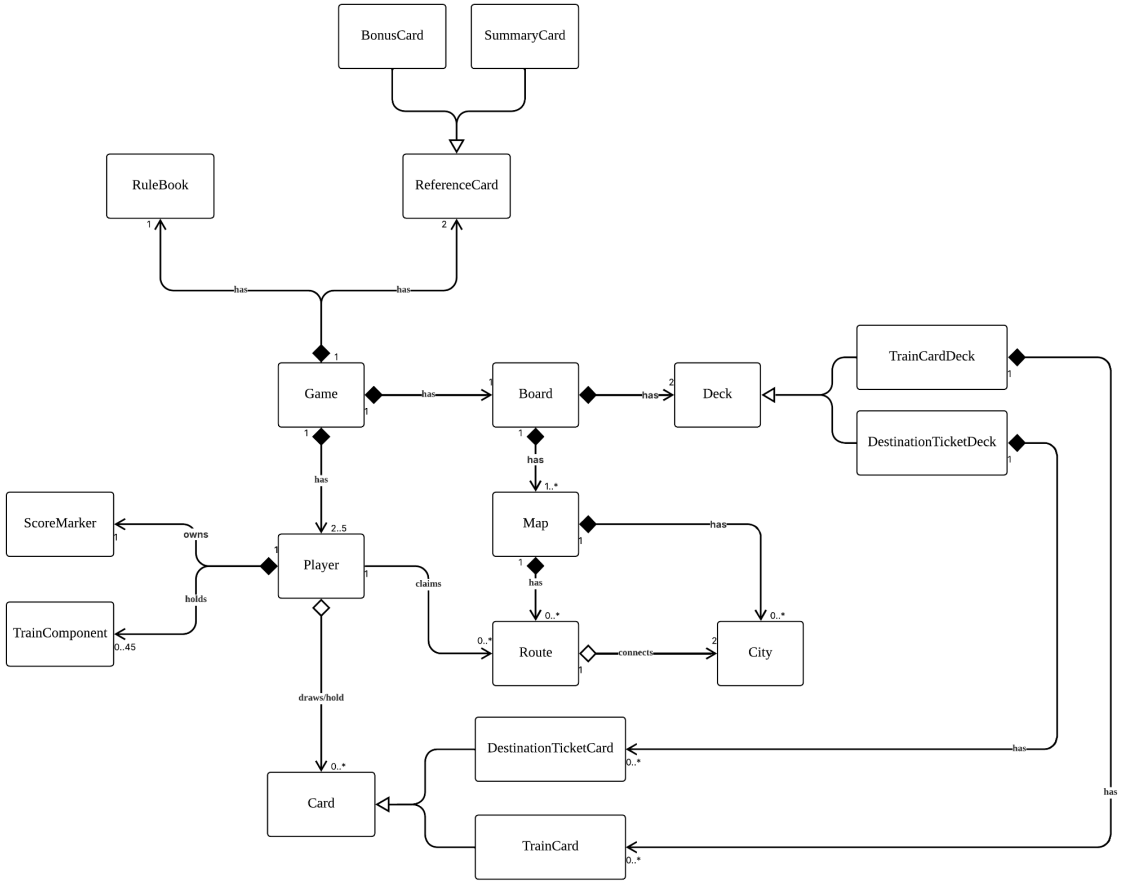


Figure 4: Discarded Domain Model Iteration 3

Alternative Logic:

This version further extends Alternative 2, introducing abstract entities for reuse and generalization. Card is abstract, generalized by TrainCard and DestinationTicketCard. Deck is abstract, generalized by TrainCardDeck and DestinationTicketDeck. ReferenceCard is abstract, generalized by SummaryCard and BonusCard. Also, Player aggregates Cards instead of specific Card types. The board composes Decks instead of specific Decks. Game also now composes RuleBook and ReferenceCards instead of attaching them to the Board. Map is now introduced and composed by the Board, and it composes Routes and Cities. Player now has ScoreMarker, and cardinalities for Game-Player (2-5) and Player-TrainComponent (0-45) are bounded.

These abstract entities reduce repetition and improve the model maintainability. Domain constraints, such as bounded numbers of Players and TrainComponents, are now reflected more clearly. Introducing a Map from the Board also improves the single responsibility principle, making the model easier to understand.

Rationale for Discarding:

1. **Over-tight composition and aggregation misuse**: Some compositions (e.g. Game → ReferenceCard, Board → Deck) may still be too restrictive. Also, the aggregation between Route and City may imply ownership that does not exist.

2. **Overengineering with ReferenceCard abstraction**: Creating an abstract ReferenceCard for SummaryCard and BonusCard may add complexity without significant benefit as these cards may not share the common properties.

3. **Cardinality refinement still needed**: Some deck-card relationships ignore the fixed number of cards in the game. Using exact bounds better reflects the game's domain rules. Similarly, the (0..*) cardinality between Map and Routes or Cities could be misleading, as it implies a Map without any Cities or Routes, which is invalid in Ticket to Ride and may confuse readers.

4. **RuleBook and SummaryCard**: RuleBook and Summary Card are to be excluded from the problem space domain model as they provide little to none concepts or relationship that is required to be handled by the system as the logic running in the system itself are based on the rule described in the RuleBook where SummaryCard are just a visualize reference to the Player. Since both of them are not relevant problem space to be solved and only represent as physical artefact of the game, hence they are excluded from the domain model.

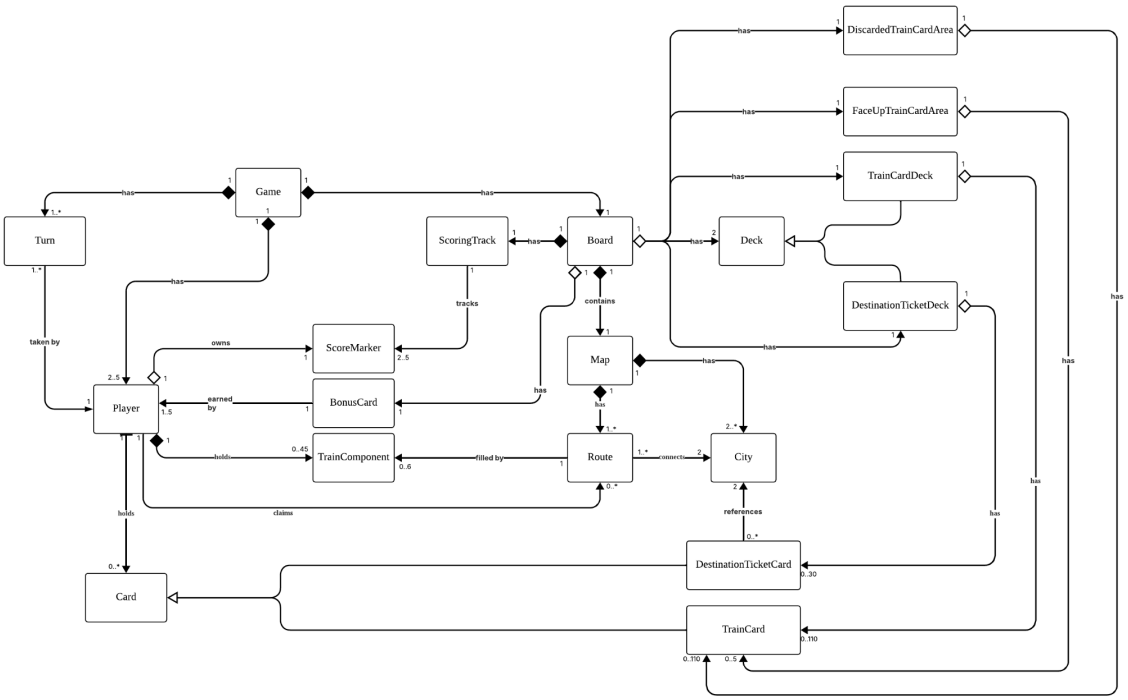


Figure 5: Domain Model Iteration 5

Alternative Logic:

This version further extends the previous domain model iteration by introducing Turn, ScoringTrack, DiscardedTrainCardArea and FaceUpTrainCardArea. Turn is added as an important domain as it represents an important lifecycle of the gameplay, where the game consists of multiple turns and each turn is taken by one player sequentially. ScoringTrack was added to show the physical tracker of scores for each player. DiscardedTrainCardArea and FaceUpTrainCardArea is further associated with the Board as it may be treated as a physical component on the board that players can view visually. RuleBook, ReferenceCard and SummaryCard are removed from the model as they serve as physical components of the game with minimal to no impact.

This allows for a clearer conceptual picture of the main domains in the game.

Rationale for Discarding:

1. **Incomplete Multiplicity**: Some of the multiplicity were partially correct without reflecting the whole game logic. For example, the relationship between Player to Card is *1* to *0…** which implies each Card will always be held by exactly one Player. However, the Card may also exist in multiple states such as in DiscardedTrainCardArea, FaceUpTrainCardArea and Deck but not always held by Player, which makes the multiplicity for the relationship between Player and Card should be *0..1* to *0…** and the same goes to other components/entities that have associations with TrainCard and DestinationTicketCard.

2. **Deck duplicated:** Lifecycle of TrainCard such as in DiscardedTrainCardArea, FaceUpTrainCardArea, TrainCardDeck are all tightly coupled to Board, which can be consolidated into TrainCardDeck holding DiscardPile and FaceUpTrainCardArea.

3. **Physical Artefacts**: ScoreMarker and ScoringTrack are only a physical artifact to reflect the scoring of each Player in the game which do not contribute or influence to the scoring logic of the game, which is not suitable as a problem space domain model entity. Map is also a physical artefact that does not contribute any meaningful lifecycle to the game, hence having a Map introduces unnecessary complexity to the model.

3.1.2 Final Domain Model Chosen

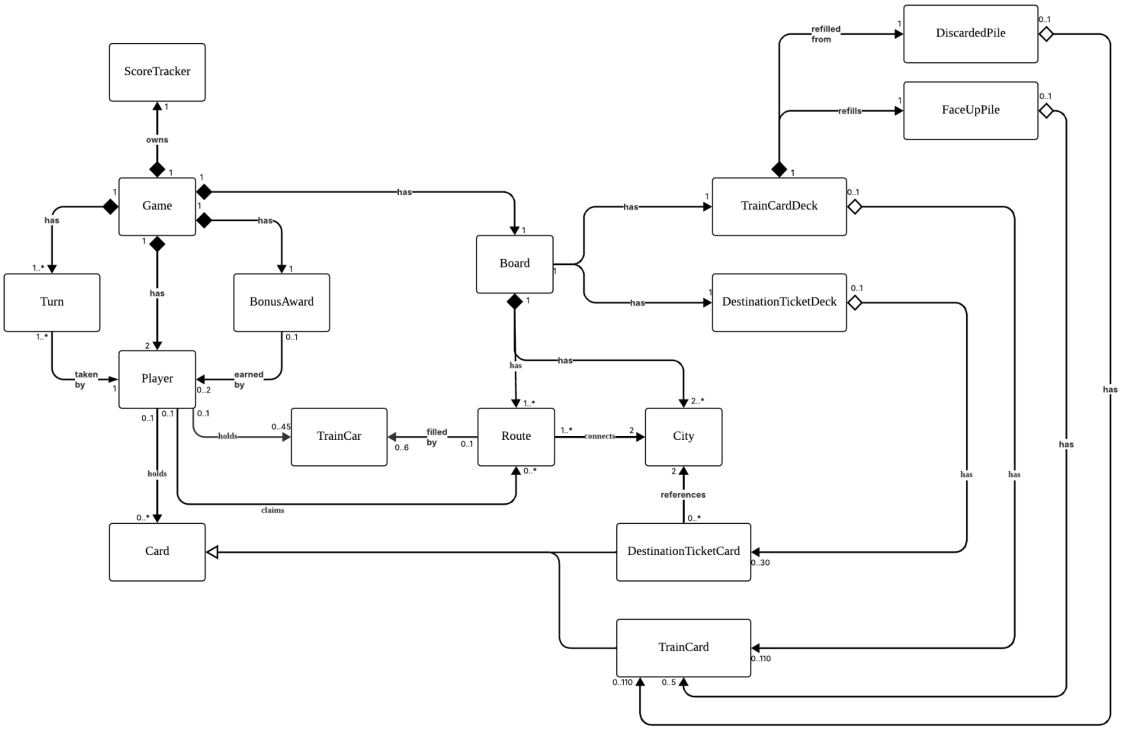


Figure 6: Final Domain Model

Rationale for chosen:

The selected domain model was chosen over the earlier alternatives because it provides a clearer and more complete representation of the Ticket to Ride gameplay. Each entity and relationship is justified through the concept descriptions and domain constraints, ensuring that the model consistently captures the key logic and rules of the game.

Earlier versions either oversimplified or overcomplicated the domain, introducing unnecessary entities and responsibilities across some concepts. This final domain model achieved a balanced level of abstraction by clearly separated core concepts like Game, Player, Board and Turn, but still models important gameplay concepts like TrainCardDeck, DestinationTicketCardDeck, FaceUpPile, DiscardedPile for card lifecycle, and Route ownerships.

The Deck abstraction introduced in previous iterations is also removed, as using a general Deck entity made it difficult to clearly enforce that exactly one TrainCardDeck and one DestinationTicketDeck exist. Since extensibility is not a concern in this domain, the abstraction added unnecessary complexity without improving clarity. The final model thus remains explicit deck entities to better reflect the game rules.

Also, although earlier iterations suggested a lack of separation between Board and Map, the final model remains only the Board as a single concept, a separate Map entity was unnecessary as it represents a static structure with no meaningful lifecycle or behaviour impact on gameplay. The Board alone is sufficient to capture the game's structure without introducing redundant complexity.

The final model also refines multiplicities by applying appropriate optionality across relationships. Earlier iterations tended to imply mandatory associations, whereas in the domain many elements (e.g., cards, routes, train cars, and bonus awards) may exist independently or transition between states. By using optionality (e.g., 0..1 where appropriate), the model more accurately reflects these dynamic states and avoids imposing unintended constraints on the relationships.

3.2 Concept Descriptions

Game Core Concepts

Concept Name	**Description**	**Relationships**
Game	Represents a single session of the Ticket To Ride game, including all participating players, components, and the progression of play from start to end. **Justification**: This concept is needed as it provides the overall context in which all other elements (e.g. players, cards, routes) exist and interact meaningfully. It has a clear lifecycle, progressing from setup through gameplay to the end of the session. Without modelling the game explicitly, it would not be possible to manage the progression, state and enforce rules consistently	● A game is played by exactly 2 Players. ● A game uses exactly 1 Board. ● A game consists of one or many Turns. ● A game has exactly 1 ScoringTracker to keep track of scores. ● A game has exactly 1 BonusAward. **Justification**: Ticket to Ride is a multiplayer game, but with a fixed 2 player range, so associating Game with Player represents participation and ensures a valid multiplayer gameplay configuration. A Game is linked with exactly one Board as each game uses exactly one Board as the play area, and it

	across an individual game session.	cannot be shared between multiple active game sessions at the same time. Also, gameplay proceeds in a structured Turn sequence, where Players take Turns sequentially one at a time. Thus, the Game is associated with Turn to ensure fairness and order. A Game is linked with exactly one ScoreTracker to ensure that all score changes are consistently recorded within the same game context. A Game has exactly one BonusAward, which the bonus score for a specific achievement (longest continuous route) is defined from game start, and will be awarded to 0 to 2 players. Composition relationship type is used between Game with all entities associated with it because these elements are strongly tied to a specific Game session, they do not exist meaningfully outside of the Game session. Their lifecycle is dependent on the Game, and they are created, used and removed as part of a single Game session.
Player	Represents a participant in a Ticket to Ride game session, who can perform actions. **Justification**: This concept is needed as it is the active person who makes decisions and performs actions in the	● A player participates in exactly 1 Game at a time. ● A player takes one and more Turns in a Game. ● A player holds 0 to many Cards. ● A player claims 0 to many Routes. ● A player holds 0 to 45

	game. Without players interacting with the game, no gameplay can occur. It also allows the modelling of ownership for several game elements (cards, routes, train cars), which is important for competing to get the highest score in the game.	TrainCars. ● A player may earn 0 or 1 BonusAward. **Justification**: Player is linked with Game (composition type) to represent participation within a specific game session. The Player is defined in the context of that specific Game session and does not exist independently within this domain. Players associated with Turn as a player can participate in many Turns in a game, reflecting an active role in the gameplay. Players are associated with Cards (TrainCards, DestinationTicketCards) because players are able to plan and execute strategies based on the card they hold. A Player is associated with Route as a player has the opportunity to claim a route at each game turn, provided that they fulfil the condition of claiming the specific route. This represents an influence on the game. This action is constrained to have sufficient and valid TrainCars, hence Player is associated with TrainCar as well. Player is associated with BonusAward as this captures the achievement of the Players during the end game, which is a mandatory outcome of the game.
Turn	Represents a player's turn in the game, during which the	● A turn belongs to exactly 1 Game.

	player performs one main action. **Justification**: This concept is needed to form an ordered flow in the progression of the game, ensuring that players take turns in sequence, making sure that actions are performed in an organised and fair order, and maintaining rule consistency.	• A turn is taken by exactly 1 Player. **Justification**: Each Turn must be associated (composition type) with exactly one Game to maintain context. And composition is used as Turns are created as a part of a Game session, their lifecycle is fully dependent on the Game. Each Turn is linked to exactly one Player to show whose turn it is to perform an action. This is to ensure a clear and traceable gameplay sequence.
ScoreTracker	Represents the system that will record each player's score throughout the game. **Justification**: This concept is needed as it represents the dynamic scoring system of the game. This concept abstracts the physical ScoringTrack and ScoringMarker that keep track of player's score into a system that consistently maintains the score progression of players, which changes over time and must be tracked consistently within a game session.	• It belongs to exactly one Game **Justification**: ScoreTracker is linked to exactly 1 Game so that it will keep track of all scoring updates in the correct Game session. This helps to maintain consistency in scoring updates to determine the final outcome of a game session. And composition is used because it is part of a Game session and is used to track scores within that specific Game, with no independent meaningful existence.

Board Structure

Concept Name	**Description**	**Relationships**

Board	Represents the physical game board that holds the map and all shared game elements used during a game session. **Justification**: This concept is needed to provide the main play area where all shared components are organised and game interactions occur. It acts as a container for all core components like card decks, cities and routes, ensuring all shared resources can be accessed.	• A Board belongs exactly to 1 Game. • A Board has exactly 1 TrainCardDeck. • A Board has exactly 1 DestinationTicketDeck • A Board has 1 to many Routes. • A Board has 2 to many Cities. **Justification**: A Board is owned by exactly 1 game with composition relationship type as this reflects that a board exists as a play area for a specific game, without game, it is meaningless. The Board organises all key game components in one place. Each component (Card Decks, Card Piles, Routes, Cities) is fixed and always present during the game, so they must be associated with the Board to reflect how the physical game is set up and played, ensuring that all shared elements are consistently managed. Composition relationship type is used with City and Route as they are structural components of the Board and exist as part of its fixed layout. They are defined as part of the Board and continue to exist as long as the Board exists, reflecting a strong whole part relationship.
Route	Represents a connection between two cities that can be claimed by a player. **Justification**: This concept is needed as it	• A Route connects exactly 2 Cities. • A Route may be claimed by 0 or 1 Player. • A Route can be filled by 0 to 6 TrainCars as evidence of a

	defines the connections between Cities that have their own properties (length, ownership, requirements) which cannot be represented by the Cities alone. Hence, it needs to be modelled as a separate entity to allow each connection to be individually tracked and managed within a game session.	claimed route. ● A Route belongs to exactly 1 Board. **Justification**: A Route must connect exactly two Cities to form a valid link in the network. A Route is associated with 0 or 1 Player to enforce game rules, where each Route can only be claimed once by exactly one Player per game session, and may not be claimed by any Player throughout the game session. Associating Route with 0 to 6 TrainCars represent the physical occupation of the Route after it is claimed, and restricted to a upper bound of 6 TrainCars due to the scoring board upper bound is 6 TrainCars. Associating Route with Board ensures the Route belongs to a specific Game. And composition relationship is used as Route is defined as part of the Board's structure; without the Board, Route has no standalone meaning within this domain.
City	Represents a named location on the board, which acts as a connection point for routes. **Justification**: This concept is needed as it defines the connection endpoints for routes. It also offers identifiable locations	● A City belongs to exactly 1 Board ● A City is connected by one or more routes. ● A City is referenced by 0 or many DestinationTicketCards. **Justification**: Associating City with exactly 1 Board

	that allow connections to be formed and referenced across the game. Without cities, it would not be possible to define journey objectives for a player to compete on.	is to ensure consistency with a specific board configuration. And a composition relationship is used as City is defined as part of the Board's structure; without the Board, City has no standalone meaning within this domain. Cities must be connected through Routes to form the game network. Associating City with DestinationTicketCard allows game objectives to be defined, as they are used as reference endpoints of a journey in DestinationTicketCards, so they need to be consistently identifiable.

Player Components

Concept Name	**Description**	**Relationships**
TrainCar	Represents a coloured train piece that a player places on their claimed routes on the board. **Justification**: This concept is needed as each player's claimed routes must be represented physically on the board map. It helps to ensure the route occupation is managed consistently. Furthermore, this concept is introduced to enforce the game rule, limiting the number of routes that a	• A TrainCar is held by exactly 0 to 1 player. • A TrainCar can be placed on 0 to 1 Route. **Justification**: Each TrainCar is tied to a single Player to show the current state of TrainCar during gameplay. A TrainCar is held by a Player when it is still in their hand, and is placed on a Route when the Route is claimed. Their placement on Routes allow the system to track claimed Routes by the Player. This ensures consistent gameplay and enforcing game rules on route occupation.

	Player can claim, where each Route must be filled by 1 to 6 TrainCar when claimed and each Player only possesses exactly 45 TrainCar at the start of the game.	Each Player will only have TrainCars of exactly one colour, which is distinct between Players, reflecting ownership.

Cards and Decks

Concept Name	**Description**	**Relationships**
Card	Represents the abstract concept of a card in the game. It acts as a superclass for all drawable cards. **Justification**: This abstract concept is needed as a common abstraction for all types of cards (TrainCards and DestinationTicketCards). Modelling it can avoid duplication of shared relationships, and provides a clear structure for the card hierarchy in the domain model.	● Card is specialised by TrainCard and DestinationTicketCard ● Card is held by 0 or 1 Player. **Justification**: Card is specialised into TrainCard and DestinationTicketCard to ensure each card type can define their own unique properties but still inheriting common behaviour for Player to hold, ensuring consistency in how cards are managed within the Game. Card is associated to 0 or 1 Player to show the behaviour of Player is capable of holding Cards, while Card also may not be held by any Player, showing game rules where Card can exist in the Deck.
TrainCard	Represents a coloured card used by players to claim routes. **Justification**: This concept is needed because players use	● A TrainCard is specialised from Card. ● A TrainCard belongs to exactly 0 or 1 TrainCardDeck. ● A TrainCard belongs to exactly 0 or 1 FaceUpPile. ● A TrainCard belongs to exactly 0 or 1 DiscardedPile.

	TrainCards as the primary source to claim routes, and each card has a state and identity that must be tracked. It allows the game to manage the availability and movement of TrainCards as ownership and gameplay actions consistently. A standard Ticket to Ride game has 110 train cards – 12 cards for each 8 colours and 14 locomotive cards.	**Justification**: TrainCard specialises from Card to show that TrainCard holds the common behaviour of Card to be drawable, while maintaining its own identity. A TrainCard is associated to exactly 0 or 1 TrainCardDeck / FaceUpPile / DiscardedPile because one TrainCard can only exists in exactly one state, either in the TrainCardDeck, in the DiscardedPile, FaceUpPile, Player at any time throughout the game session. This is to ensure accurate tracking of availability and ownership. Aggregation relationship type is used with TrainCardDeck, FaceUpPile, and DiscardedPile as these are containers of the TrainCards. But, since TrainCards can move between different containers during gameplay, their lifecycles are not owned by any single container.
DestinationTicketCard	Represents a hidden objective specifying two cities and a point value. The player who holds the destination card gains the points if they successfully connect the two cities with a continuous chain of claimed routes by the end of the game; otherwise the player loses the points. **Justification**: This concept is needed as it defines specific objectives	● A DestinationTicketCard is specialized from Card. ● A DestinationTicketCard belongs to 0 or 1 DestinationTicketDeck. ● A DestinationTicketCard references exactly two Cities. **Justification**: DestinationTicketCard specialises from Card to show that DestinationTicketCard holds the common behaviour of Card to be drawable, while maintaining its own identity.

	that players are required to achieve. It helps to allow the game to track which objectives exist, which are assigned to players and whether they are completed by the game end. This is necessary to model win conditions of the game.	A DestinationTicketCard is associated with 0 or 1 DestinationTicketDeck because it can only exist at one state at any time of the game session, either remain in the DestinationTicketDeck to be drawn by Player or in Player to achieve the hidden objective. This is to ensure accurate tracking of availability and ownership. Aggregation relationship type is used with DestinationTicketDeck as it is a container of the DestinationTicketCards. But, since DestinationTicketCards can move between different containers during gameplay, their lifecycles are not owned by any single container. DestinationTicketCard references exactly two Cities to define the objectives that Players aim to achieve clearly.
TrainCardDeck	Represents a collection of train cards that players can draw from during the game. **Justification**: This concept is needed to manage the pool of train cards that are available to be drawn. It helps allow the game to track which cards are still available and separate them from those in players' hands, face-up pile, or in the discarded card pile, showing the lifecycle of TrainCards.	● A TrainCardDeck belongs to exactly 1 Board. ● A TrainCardDeck manages 0 to many TrainCards ● A TrainCardDeck contains 1 FaceUpPile ● A TrainCardDeck contains 1 DiscardPile **Justification**: The TrainCardDeck is associated with a Board ensuring it belongs to a specific game. A TrainCardDeck is associated with 0 to many TrainCards to show the collection of TrainCards that is

		pending and available to be drawn by Players, and to be refilled to the FaceUpPile, and to be refilled by the DiscardedPile when insufficient. An aggregation relationship type is used as TrainCardDeck is a container to hold and manage TrainCards without owning their lifecycle, TrainCards can move between different containers during gameplay, so their existence is not dependent on a single TrainCardDeck. It contains only the TrainCards that are still in the deck available to draw, separating them from Cards already held by Players, displayed in FaceUpPile, or in the DiscardPile, reflecting the real gameplay mechanics and ensuring all card states are not duplicated. Composition relationship type is used with FaceUpPile and DiscardedPile as they are integral parts of the TrainCardDeck's lifecycle within a Game session. They do not exist independently outside the context of the TrainCardDeck. They are created, maintained, and managed as part of it. Their existence depends on the TrainCardDeck's operations (e.g. refilling, reshuffling), reflecting a strong whole-part relationship.
DestinationTicketDeck	Represents a collection of destination ticket cards that players can draw from during the game. **Justification**: This concept is needed to manage the pool of objective	● A DestinationTicketDeck belongs to exactly one Board. ● A DestinationTicketDeck contains 0 to many DestinationTicketCards. **Justification**: A DestinationTicketDeck is

	cards that have not yet been assigned to players and control their distribution to players, showing the lifecycle of DestinationTicketCard.	associated with a Board ensuring it belongs to a specific game. A DestinationTicketDeck is associated with 0 to many DestinationTicketCards to show the collection of DestinationTicketCards that is pending and available to be drawn by Players, reflecting the game state correctly, showing which objectives are still available for Players to draw during gameplay. And aggregation relationship type is used as DestinationTicketDeck is a container to hold and manage DestinationTicketCards without owning their lifecycle, DestinationTicketCards can move between different containers during gameplay, so their existence is not dependent on a single DestinationTicketDeck.
FaceUpPile	Represents the visible train cards available for players to draw. **Justification**: This concept is needed to show which TrainCards are immediately available to players, allowing for strategic player choice to be made. Its state changes as cards are drawn and replaced, capturing the card availability in the domain.	● A FaceUpPile belongs to exactly 1 TrainCardDeck. ● A FaceUpPile holds up to 5 TrainCards. **Justification**: A FaceUpPile is dependent on a specific TrainCardDeck to allow for proper refills from the specific TrainCardDeck when visible TrainCard is insufficient. And composition relationship type is used as FaceUpPile depends on the TrainCardDeck for their existence and lifecycle, because their contents are derived from TrainCardDeck operations. A FaceUpPile is limited to display at most 5 face-up TrainCards for

		Players to draw, while it is still possible to display fewer TrainCards when there are not enough TrainCards available in the TrainCardDeck or DiscardedPile to refill the FaceUpPile. This is necessary to reflect the gameplay constraints. And aggregation relationship type is used as FaceUpPile is a container to hold and manage TrainCards without owning their lifecycle, TrainCards can move between different containers during gameplay, so their existence is not dependent on a single FaceUpPile.
DiscardedPile	Represents the pile of used train cards that may later be reshuffled into the TrainCardDeck. **Justification**: This concept is needed to separate train cards that have been played from those still available in the deck or face-up pile. It helps allow the game to track discarded cards and maintain the state of the card system clearly.	● A DiscardedPile belongs to exactly 1 TrainCardDeck. ● A DiscardedPile contains 0 to many TrainCards. **Justification**: A DiscardedPile is associated with exactly 1 TrainCardDeck because it reshuffles to refill TrainCards to the specific TrainCardDeck when needed, allowing gameplay to continue. And composition relationship type is used as DiscardedPile depends on the TrainCardDeck for their existence and lifecycle, because their contents are derived from TrainCardDeck operations. A DiscardedPile is associated with 0 to many TrainCards to hold discarded TrainCards in a pile clearly to ensure the model reflects the real game setup and to allow TrainCards be managed consistently. DiscardedPile is a container to hold and manage TrainCards without owning their

		lifecycle, TrainCards can move between different containers during gameplay, so their existence is not dependent on a single DiscardedPile.

Special Components

Concept Name	**Description**	**Relationships**
BonusAward	Represents a special bonus awarded to the player(s) with the longest continuous route at the game end. **Justification**: This concept is needed to ensure the meaningful end-game bonus is explicitly captured in the domain, while keeping the concept separate from the system that tracks player scores.	● A BonusAward belongs to exactly 1 Game. ● A BonusAward may be earned by 0 to 2 Players throughout a game session. **Justification**: BonusAward belongs to a specific Game session to represent its existence throughout the game lifecycle, even though it is only assigned to player(s) at the end of the game session. This is to ensure the system can consistently apply the end game scoring logic. Composition relationship type is used as BonusAward is defined and evaluated within a specific Game session and does not exist meaningfully outside it. It needs to be associated with Players as it represents a reward earned by those who meet specific conditions. The multiplicity reflects that it is unclaimed during the game and is earned by one or two players at the end (e.g. multiple players share the longest route)

3.3 Assumptions and Domain Constraints

3.3.1 Game Setup and Scope

- The game is modelled specifically for a basic version of the Tickets to Ride game, and does not include a complex multiplayer option for more than 2 players.
- The game only consists of a North America version, restricting to one specific set of fixed Routes and Cities, and will not be considering different versions which may introduce different sets of routes and cities.
- Before the game setup phase, each Player holds 0 Cards.
- After the game setup, each Player is dealt with 4 TrainCards and 3 DestinationTicketCards, and must keep at least 2 of the 3 DestinationTicketCards.
- Each Player begins with exactly 45 TrainCars, and will not be able to add additional TrainCars after the game starts.

3.3.2 Turn and Gameplay Rules

- Player take turns in a fixed clockwise direction determined at game start.
- In each Turn, a Player performs exactly one main action of drawing TrainCards, claiming a Route, or drawing DestinationTicketCards.
- A Player may draw up to two TrainCards in a Turn. If a face-up locomotive card is drawn, the Player is not allowed to draw a second TrainCards and the turn is considered as ended.

3.3.3 Card and Resource Constraints

- A TrainCard exists in exactly one location (TrainCardDeck, FaceUpPile, a Player, DiscardedPile) at any time throughout a game session.
- A DestinationTicketCard exists in exactly one location (DestinationTicketDeck, a Player) at any time throughout a game session.
- The number of TrainCards in TrainCardDeck may change during gameplay due to drawing and reshuffling of TrainCards, but it never exceeds 110 TrainCards.
- The number of DestinationTicketCards in DestinationTicketDeck may change during gameplay due to drawing of DestinationTicketCard, but it never exceeds 30 DestinationTicketCards.
- If three or more locomotive TrainCardscards appear in the FaceUpPile, all face up TrainCards are discarded and replaced from TrainCardDeck.

3.3.4 Route and Game Structure Constraints

- Each Route connects exactly two cities, and there is no route between a city and itself (self-loop).
- Each Route can only be claimed by at most one Player during a Game session, and once claimed, it cannot be unclaimed or reassigned to another Player.

- Each DestinationTicketCard references exactly two distinct Cities, the origin and destination of the journey, and the connection is undirected.
- A DestinationTicketCard is considered as completed if a continuous path exists between the two cities in either direction through the Player's claimed Routes.

3.3.5 End Game and Scoring Rules

- When any Player's remaining TrainCars are left with 2 or fewer at the end of their Turn, the final round is triggered.
- After the final round is triggered, each Player takes exactly one final Turn before final scoring is calculated.
- The Player (s) with the longest continuous path at the end of the game are awarded a BonusAward.
- The Player with the highest final score is selected as the winner. If a tie exists, the Player with the highest number of completed DestinationTicketCard is selected as the winner. If the tie still persists, the Player who is awarded the BonusAward will be selected as the winner if the other Player is not awarded the BonusAward, else the game is considered as a draw.

4 Dynamic Model

4.1 Scenario: Claim a Route (Standard Turn)

Scenario Description:

This scenario describes how the system handles a player attempting to claim a specific route during their turn.

This scenario starts with the player selects a target route and the set of train cards they wish to use. Then, the system first checks if this is the player's first action of the turn; if they have already drawn a card, the claim is rejected to prevent illegal multi-action turns.

If the turn state is valid, the system then verifies if the route can be claimed, specifically verifying that the route is not already owned, the player has enough train cars, and that the selected cards match the route's length and colour requirements, including the use of locomotive cards where applicable.

If any requirement fails, the system will notify the player of the failure. The player may then choose another valid action, as their turn has not ended.

If the validation succeeds, the system updates the game state by deducting the used cards and train pieces from the player, moving the used cards to the discard pile, assigning ownership of the route, and applying the corresponding score.

Finally, the system completes the player's turn and transitions to the next player.

This scenario focuses only on the standard turn flow and does not include special conditions such as the last round triggered.

Sequence Diagram:

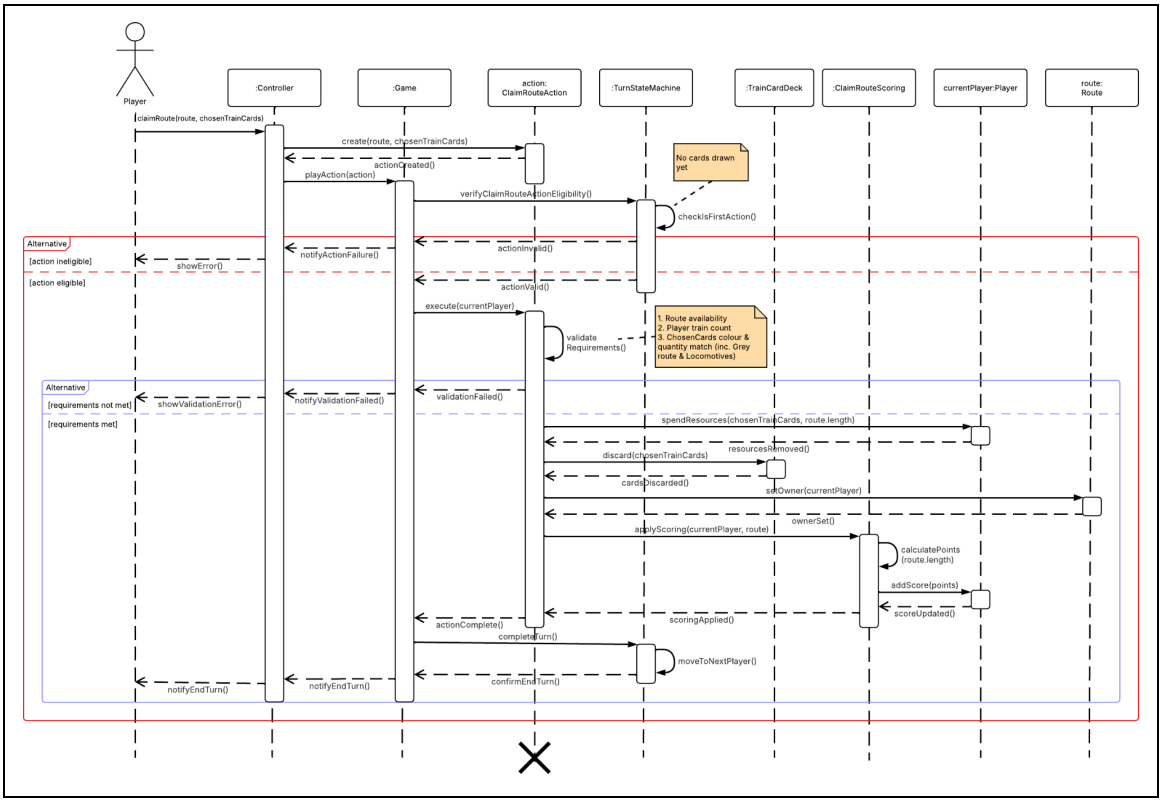


Figure 7 : Sequence Diagram 1

4.2 Scenario: End-game Final Scoring and Winner Selection

Scenario Description:

The scenario starts with the final Player completing the last turn after the end-game condition has been triggered and all other Players have taken their final round. The Game will receive a *GAME_OVER* signal after the final Player succeeds the final action and transitions into the end-game final scoring phase according to the rule.

The system recounts the scores for each player. The system then updates the player score depending on the completion of DestinationTicketCard held by Players, and updates each Player with the number of successfully completed DestinationTicketCards. The system also evaluates the longest continuous path and identifies the Player(s) who are awarded the bonus.

After all end-game scoring has been applied, the Game determines the winner. The player with the highest score is selected as the winner. If both Players have the same highest score, the Player with the higher number of completed DestinationTicketCards is selected as the winner. If there is still a tie, and only one Player is awarded the LongestContinuousPath bonus score, the Player with the award is selected as the winner, else the game is declared as a *DRAW* if both Players are awarded the bonus score.

Sequence Diagram:

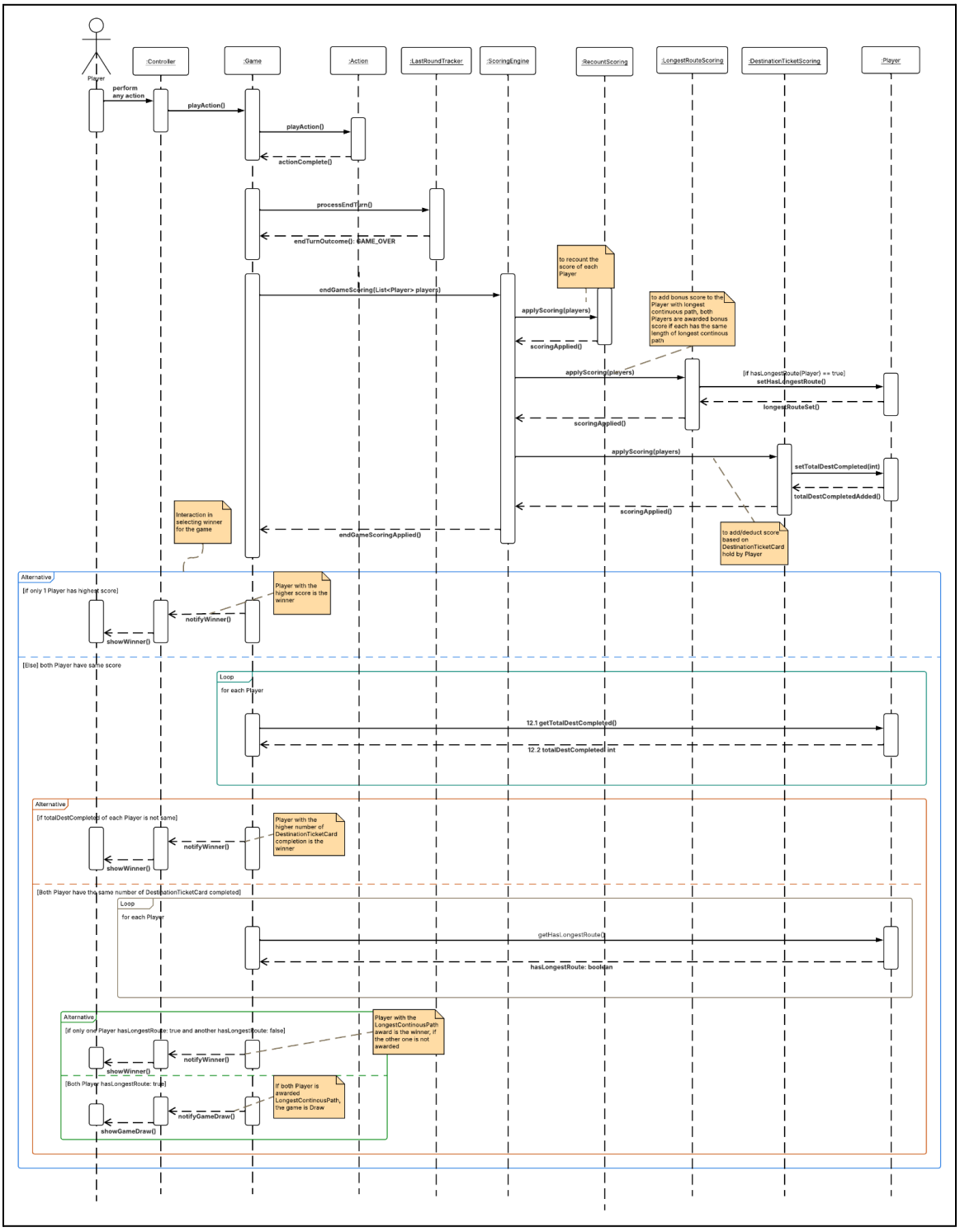


Figure 8 : Sequence Diagram 2

5 Supplementary Documentation

5.1 Additional Documentation for Scenario 4.2

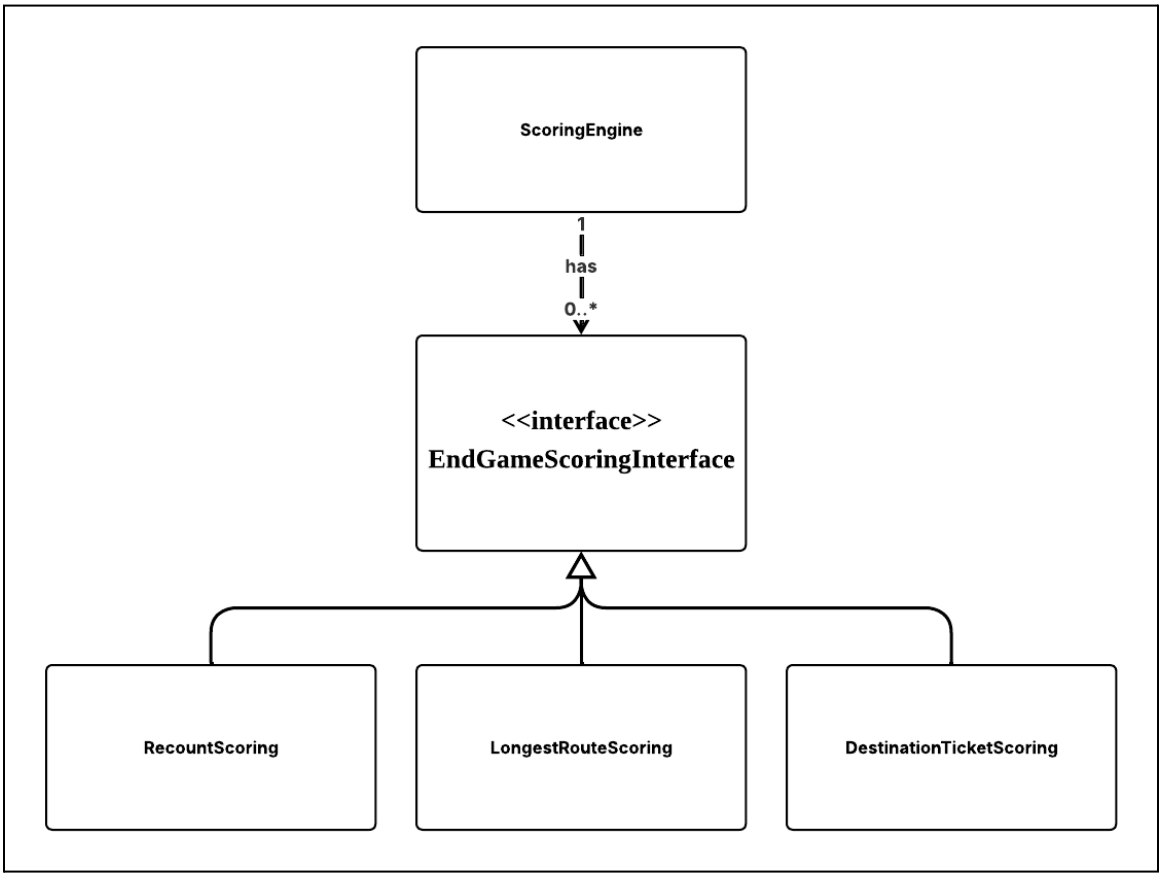


Figure 9 : Sequence Diagram 2 Support Diagram

ScoringEngine conceptually holds a list of EndGameScoringInterface which are abstractions of end-game scoring logic. The interface and its implementation are modeled at a conceptual level to support extensibility in ScoringEngine while also enforcing Single Responsibility

5.2 Contributor Analysis



Figure 10 : Sprint 1 Contributor Analysis

6 AI Declaration

The team used AI in the following ways:

1. Used ChatGPT to aid in rephrasing certain sentences to improve clarity and ensure the intention was clearly expressed, the rephrased sentences are further edited by the team to meet the expectations.